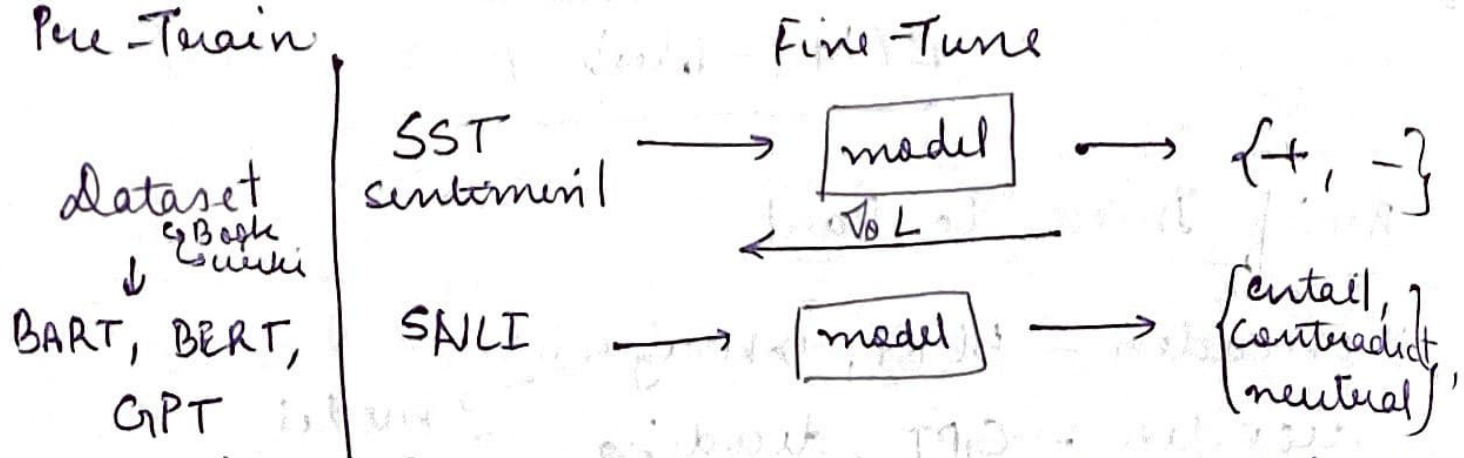


11 Brief Intro to Bart

- encoder - BERT, tokenizers
- decoder - GPT, decoding → model agnostic
- enc-dec - BART, T5
- BERT is good at comprehension tasks like QA because of its bidirectional nature.
- GPT is good at text generation since it is unidirectional nature.
- BART → bidirectional autoregressive transformers
 - ↳ mask a set of words, then try to predict it.
- Vanilla transformer architecture + GELU
 - enc. takes the corrupted sequence.
 - dec. predicts entire seq. and calculate
 - the loss over the entire sequence.
 - like a denoising autoencoder.

12 GPT2 - Prompting the Models

- using any of the pre-trained models, for downstream tasks require us to independently fine-tune the parameters of the model



- we make the copy of pre-trained model for each task & fine-tune for each task.
 - fine-tuning large models is costly, still req. 1000s of samples to get good performance. Some tasks might not have enough labelled samples. But, this is not the how humans learn the language. We don't need supervised fine-tuning.
 - do multiple tasks with zero-shot (w/o explicit supervision), few-shot, full fine-tuning.
- unsupervised learners $\Rightarrow$ GPT-2

$p(\text{output} | \text{input})$

$\downarrow$ LM change in prompt

$p(\text{output} | \text{input}, \text{task})$

added as an instruction during inference

- to get a good performance, we need to scale both model size & data size
- | | | |
|--------------------|--|----------------------|
| GPT
110 million | layers 4x
$\rightarrow$
params 10x | GPT 2
1.5 billion |
|--------------------|--|----------------------|

- dataset $\rightarrow$ WebText (from reddit) - diverse & good quality of data. But what abt. "translate" tasks? There were some naturally occurring demo of Eng to Fr. & vice-versa in the training set!

$\rightarrow$ 40 GB (8 million web-pages)

$\rightarrow$ Byte-level BPE tokenizer

$\rightarrow$ Vocab $\rightarrow$ 50,257 tokens

$\rightarrow$ GPT variants

params	layers	dmodel
117 M	12	768
345 M	24	1024
762 M	36	1280
1542 M	48	1600

$\rightarrow$ context window - 1024

$\rightarrow$ batch size - 512

$\rightarrow$ causal lang. modelling

$\rightarrow$ performance on 0-shot transfer - measure the performance of the pre-trained model on domain-specific benchmarking dataset.

$\rightarrow$ measure perplexity $\Rightarrow$ how well it is predicting as compared to actual predicⁿ.

$\rightarrow$ charac. level LM $\rightarrow$ measure BPC

$\rightarrow$ open-ended cloze task $\rightarrow$ measure accuracy

$\rightarrow$ doing quite well with zero-shot itself.

$\rightarrow$ promising direcⁿ, but still very far from SOTA models fine-tuned for each task.

- 1.5B params model still underfits the training dataset
- LLM are unsupervised multitask learners

13 Choices that affect model performance

- comparison of GPT / BERT_{base} / BERT_{large} on various tasks
- why 1 model is better than others?
 - is it dataset size?
 - is it training obj.
 - size of model (params)?
 - training a model longer?
 - is it the way how we fine-tune?
- we have to conduct experiments by keeping 1 aspect of the pipeline constant & vary the others.
- Scale → small, medium, large
 - pre-training → wiki, books, web crawl
 - pre training HPT → train steps, learning rate, optimizer
 - fine-tune → choices used
- Wishlist -
 - can we have the same obj. for both pre-training & fine-tuning stages?
 - as large as possible dataset?
 - study by T5 paper - comp. analysis.

- Idea of T5 $\rightarrow$ formulate all NLP problem as Text-in & text out.
- T5 $\rightarrow$ Text to Text Transfer Transformer applies to all the tasks $\rightarrow$ summary, QA, NLI, etc. This is how humans will answer as well. It is given a task-specific prefix prepended to the input context.
- It looks like new over old. looks same both during pre-training as well as during fine-tuning.
- Dataset $\rightarrow$ C4 (Colossal Clean Crawled Corpus)
 - $\rightarrow$ 750 GB, 300 M docs, 156 B tokens
 - $\rightarrow$ mainly English
 - $\rightarrow$ Vocab size - 32 K
 - $\rightarrow$ sentence piece tokenizer

L4 Baseline Model

- enc. - dec. like BART, takes corrupted input & then predict.
- it has 12 enc. & 12 dec. model. 2x params as compared to BERT base.
- denoising objective - analogous to MLM, replace a random span of tokens taken by a single special token

- at end $\langle z \rangle$ to show task has been completed.
- advantage of such a obj. it reduces the training time.
- pre-training

$T = 512$ with corrupted noise

no. of training steps $= 2^{19} = 524,288$

total no. of pre-training tokens $= 2^{19} \times 2^{16} = 2^{35} = 34 \text{ B}$

$(B \times T)$
 128×512

it is actually much smaller than actual data. BERT had 137 B & RoBERTa had 212 T tokens.

reasonable for a baseline model & helps in experimenting.

Optimizer - Adafactor (req. less memory storage than Adam)

Learning Rate Scheduler - inverse sqrt scheduler

$(\triangleleft)$
 $\left(\frac{1}{\sqrt{\max(n, k)}} \right)$

$n \rightarrow$ training step

$k \rightarrow$ warm-up $= 10^4$

- pre-training
enc.-dec + 220M params + denoising objective
+ C₄ dataset (up till 34 B tokens)
 2^{19} steps

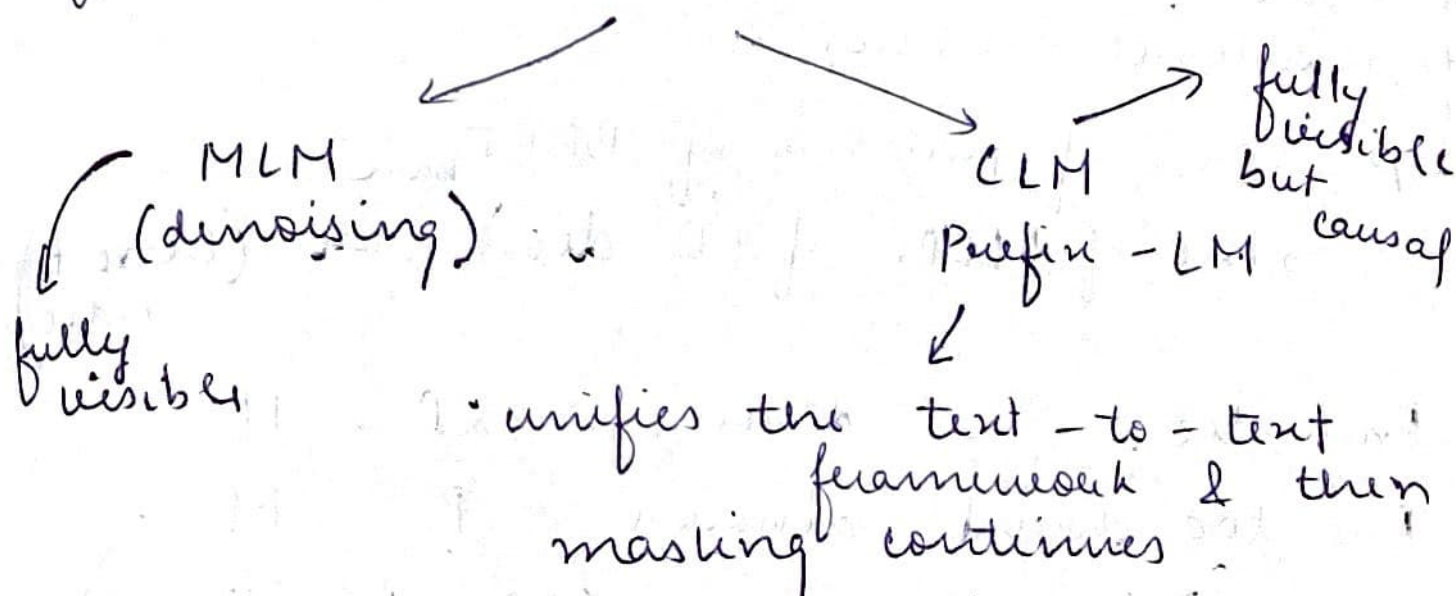
- fine-tuning — GLoVE, Super GLoVE, etc.
- 2^{18} steps ≈ 17 B tokens
- dataset similar to pre-training $\eta = 0.001$

$\overbrace{3}^{\text{talk: Translate to Russian}}$
 $\overbrace{5}^{\text{EM}}$
 $\overbrace{5}^{\text{Rus.}}$
 $\} 13 \text{ tokens}$

- then evaluate on the validaⁿ task
- this process was done - 10-times from scratch (to compute avg.)

15 Experiment with different architectures

- different model choices



during pre-training also, prefix-LM was used.

- In prefix-LM, we want the input seq. to be attended bidirectionally & task output seq. autoregressively.

Enc. $\rightarrow$ for input sequence
 Dec. $\rightarrow$ for output sequence.

- We can also use the decoder with the prefix-mask instead for causal mask by pre-pending the task to the output seq.
- the baseline model has enc. dec. . Have to compare this with enc. only or dec. only models.
- We could choose P (no. of params) in a model or amount of compute M (in FLOPs) req. to process input & target seq.
- L enc. + L dec. will have same params as a LM with $2L$ layers.
But computational cost M is same for $(L \text{ enc.} + L \text{ dec.})$ model with $(L \text{ layered})$ model.
- performance metrics

$P \rightarrow$ no. of params of BERT_{base}
 $M \rightarrow$ no. of FLOPs of $(L \text{ dec.})$ or $(L \text{ enc.} + L \text{ dec.})$

Enc. - dec.	denoise	$2P$	M
Enc. dec, shared	denoise	P	M
$\hookrightarrow$ (comparable to BERT _{base})			
Enc. dec, 6 layer	denoise	P	$M/2$
BERT _{base}	denoise	P	M
Prefix LM	"	P	M

clearly enc. - dec. shared wins profoundly

compare

- even if the obj. ~~was~~ changed from denoising to prefix LM, but results downgraded only.

16 Experiment with unsupervised Objectives

- denoising obj. worked better on downstream tasks.

	Input - Thankyou	
Prefix LM	Input Thanky -	Target
Desuffle	- - Ty - - -	- - - - -
Bert style	Ty - - < > - - w	Ty - - - - -
MASS	Ty < > < > - - < >	Ty - - - - -
(masked seq. to seq)		Ty - - - - -
Random span	Ty < x > - < y >	< x > - < y > - - < z >

- both prefix LM & Bert style were better at translaⁿ task. Desuffle was always pass.
- Overall Bert style was good.

→ mask → what %

→ replace

→ mask & drop completely

→ conti mask

- all variants perform similarly. Dropping leads to slightly better on GLUE. Replace by a sentinel-token performs well.
- What is good corruptⁿ rate?
 - 10% 15% 25% 50%
 - ↳ by default fine, not much diff.
- single, span len what to choose?
 - 2. 3 5 10
 - ↳ outperforms others

L7 Effect of size of pre-training dataset

- C4 (clean / filtered) dataset - 745 GB (base)
- C4 unfiltered - 6.1 TB - poor
- Real news - 35 GB
 - ↳ very much high quality, did well
 - quality is very impt. for good perform.
- Wiki - 16 GB
- Wiki + Bookcorpus - 17 GB
- much smaller data, but if pre-training has domain-specific knowledge then performance ↑. Hence, diversity is very impt & increase the amount of domains.
- how does repeating the example in pre-training help? → very size $2^{23} - 2^{29}$.

- always preferable to share new data w/o any repeat. Othw. the model would overfit the training data.

L8 Fine Tuning Strategies

- what is right fine-tuning strategy?
- so far what we have seen is fine tune on individual tasks

2^{19}

pre-train

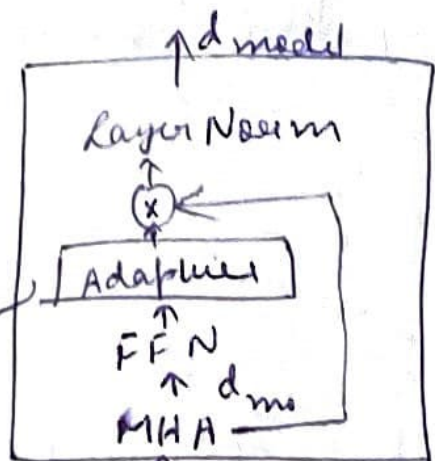
- fine-tuning all params is computationally very expensive.
- Adaptive layer

2

2^{18}

fine-tune

params is computationally

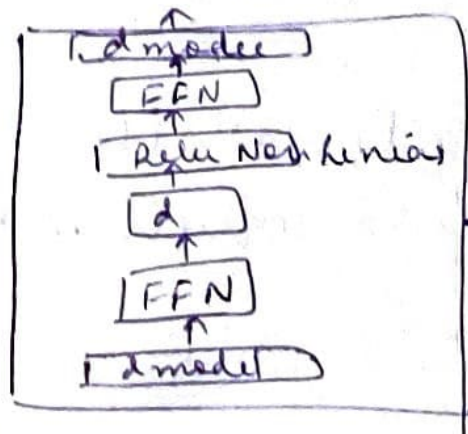


→ make sure that input & output sizes are constant

→ dense - ReLU - dense layer with dimension d with the residual connection.

→ during fine-tune only the Adapter & layer norm params are updated (only to 0.5 - 8%).

→ we can vary d & its performance on down-stream tasks



- as $d \uparrow$, performance betters, but still not comparable to full fine-tuning.
- Gradual unfreezing
 - freeze all the params of all layers
 - gradually unfreeze top (12th layer).
 - unfreeze happens simult. for k^{th} layer of enc. & dec.
 - fine tuning - 2^{18}
 - no. of layers - 12
 - 12 episodes of unfreezing
 - unfreeze happens every $\frac{2^{18}}{12}$ steps starting from the top layer.
 - speeds up the training process.
 - we can surely tune the unfreezing schedule. $\Rightarrow$ might give better results

L9 Multi-Task Learning

- earlier, we fine-tuned the pre-trained model independently on all the downstream tasks.
- why not add quality datasets in the pre-training stage itself? $\Rightarrow$ which are obs. very high quality?

- The unified T5 framework allows us to pre-train & fine-tune a single model for all tasks.
- multi-task learning

C4 + CoRe + SCoRe + Squad ... all

in pre-training $\rightarrow$ denoising objective!
with 220 M params

train for $(2^{19} + 2^{18})$ steps

- how to combine?
 - $\rightarrow$ simply concatenate all of them
 - $\rightarrow$ proportional mixing (should be in fair proportion)
- $k \rightarrow$ artificial limit to all datasets
 $N \rightarrow$ tasks

original no. of samples for each task

$e_n, n \in (1, 2 \dots N)$

probab. of a sample from m^{th} task during training is

$$x_m = \frac{\min(e_m, k)}{\sum \min(e_n, k)}$$

$\rightarrow$ use T mixing

use $(e_m)^{1/T}$ where T is temp.

$T \uparrow$, distribuⁿ uniform, else, winner takes it all

- after doing all this also, the baseline (pretraining + fine tuning) uses other examples proportional and temp. scaled
- multi task learning degrades
- you can do, multi-task learning & then do fine-tuning dataset specific ^(2'9) (2'8)
- relaxed multi task learning

→ we can take the best checkpoint for each task in the joint-fine tuning which is happening after multi-task!

- multi-task learning variants —

1. multi pre-training fine-tuning
 all all

2. leave 1-out pre training add 1
 except 1 here.

3. supervised pre-training fine-tune
 pre training except C4 add all
 not C4.

- but after all this also, (multi task pre-training + fine-tuning) all model was almost equal to $\leq$ unsupervised pre-training + fine-tuning.
- supervised pre training helps specifically on translation tasks.

L10 Pushing the limits

- what if you have 4x compute? we know scaling the params of model improves the performance.
- or build 4 models (ensemble learning)
- or just train for longer time period. Or increase the batch size
- increase the batch size $\uparrow$ the performance than baseline. Scaling the no. of params led to better results. Ensemble also gave good performance.

T5 $\rightarrow$ arch. $\rightarrow$ enc. dec

obj. $\rightarrow$ denoising

dataset $\rightarrow$ C4

train strategy $\rightarrow$ multi-task
pre training

model size $\rightarrow$ bigger the better

train steps $\rightarrow$ longer the better

pre train steps $\rightarrow 2^{20}$ steps

batch size $\rightarrow 2048$

tokens $\rightarrow 1T$

thus, it beats the SOTA models except for translaⁿ tasks.